

A 3.3.5 Database Views (HL Only)



A 3.3.5 Describe different database views. (HL only)

Virtualized and materialized database views offer flexible and performant data access. Virtual views provide dynamic, logical perspectives for flexibility and security. Materialized views store pre-computed results for faster reads. Understanding their HL-level nuances and trade-offs is crucial for efficient database design and application development.

Understanding Database Views (HL Only)

Introduction to Database Views

In database management, views provide a powerful mechanism for simplifying data access, enhancing security, and promoting data independence. As an IB Computer Science HL student, understanding the intricacies of different types of database views is crucial. This section aims to explain virtual and materialized (snapshot) views and the various benefits they offer.

At its core, a database view is a virtual table whose content is defined by a query run against the underlying base tables. Unlike base tables, views do not store data physically; instead, they act as stored queries. When you query a view, the database system executes the underlying query and presents the result as if it were a real table.

This chapter will delve into two primary categories of views: virtual and materialized views (sometimes called snapshot views). We will explore their definitions, functionalities, and the specific advantages they bring to database design and implementation.

Virtual Views

Definition and Operation

A **virtual view** is the most common type of database view. It is a named SQL query stored in the database schema. When a user queries a virtual view, the database management system (DBMS)

dynamically executes the stored query against the base tables and returns the result set. No data is physically stored as part of the virtual view definition.

Consider a scenario where you have two tables: **Customers** (with columns like **CustomerID**, **Name**, **City**) and **Orders** (with columns like **OrderID**, **CustomerID**, **OrderDate**, **TotalAmount**). If you frequently need to see a list of customer names along with the total number of orders each customer has placed, you could create a virtual view that encapsulates the necessary **JOIN** and **COUNT** operations.

```
CREATE VIEW CustomerOrderCounts AS
SELECT c.Name, COUNT(o.OrderID) AS NumberOfOrders
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.Name;
```

Whenever you query **CustomerOrderCounts**, the DBMS will execute this **SELECT** statement against the **Customers** and **Orders** tables and present the result.

Benefits of Virtual Views

Virtual views offer several significant advantages in database management:

1. Hiding Data Complexity:

Views can significantly simplify the way users interact with the database by abstracting away complex underlying structures and relationships. In intricate database schemas with numerous tables and joins, views can present a more focused and user-friendly perspective on the data. For instance, a view can combine data from multiple tables and present it as a single, logical entity, hiding the complexities of the joins and the underlying table structure. This aligns with the computational thinking concept of **abstraction**, where complex systems are simplified by focusing on essential elements. Users interacting with the **CustomerOrderCounts** view, for example, don't need to understand the underlying **JOIN** condition between the **Customers** and **Orders** tables.

2. Data Consistency:

Views can help enforce data consistency by presenting a standardised, controlled way to access information. If certain calculations or filtering criteria must be applied consistently across multiple queries, encapsulating them within a view ensures that the same logic is used every time the view is accessed. This reduces the risk of errors and inconsistencies that might arise if individual users were to write their own queries. For example, a view could always show the 'active' customers based on a specific status field, ensuring that all reports and applications using this view only consider the relevant customer set.

3. Data Independence:

Virtual views provide a degree of **data independence**. If the underlying structure of the base tables changes (e.g., columns are renamed, new tables are introduced, or tables are split), as long as the view's defining query can still produce the same logical result, applications and users interacting with the view might remain unaffected. The database administrator can modify the view's query to accommodate the changes in the base schema without requiring modifications in all the applications that use the view. This separation between the logical view of the data and the physical storage enhances the maintainability and flexibility of the database system.

4. Performance Considerations:

The impact of virtual views on performance can be nuanced. On the one hand, views themselves do not incur storage overhead as they only store the query definition. On the other hand, every time a virtual view is queried, the underlying query is executed, which can potentially impact query performance, especially for complex view definitions involving multiple joins or aggregations. However, modern DBMS often employ query optimisation techniques that can sometimes mitigate these performance overheads by efficiently rewriting queries that involve views. It's important to consider the complexity of the view's query and the frequency of its use when evaluating potential performance implications.

5. Query Simplification:

Virtual views can significantly simplify complex queries. Frequently used joins, selections, projections, or aggregations can be pre-defined in a view, allowing users to retrieve the required information with simpler **SELECT** statements against the view. This improves code readability, reduces the chances of errors in writing complex queries, and can enhance developer productivity. For instance, instead of repeatedly writing the **JOIN** between **Customers** and **Orders** tables, users can simply query the **CustomerOrderCounts** view to get aggregated information.

6. Read-Only or Updatable Data:

In many cases, virtual views can be used to update data in the underlying base tables. However, the updatability of a view depends on several factors, such as the complexity of the view's defining query. Generally, views based on a single base table, without aggregations or **GROUP BY** clauses, are often updatable. Updates made through such views are directly reflected in the base table.

Views involving joins or aggregations are typically **read-only**, meaning you cannot directly modify the base tables through them. This is because the DBMS might not be able to unambiguously determine which base table and which rows should be updated based on a modification to the view's result set. By controlling which views are updatable, database administrators can maintain data integrity and enforce business rules.

7. Security:

Virtual views provide a powerful mechanism for enhancing database security. You can restrict users' ability to see or modify sensitive data in the underlying base tables by granting users access only to specific views. A view can be designed to show only a subset of columns or rows from a table,

effectively implementing **data masking** and **row-level security** without altering the base table's structure. For example, a view could expose only non-sensitive customer information to certain user roles while hiding personal identification numbers or financial details. This principle aligns with the ethical considerations surrounding data privacy and security.

Materialized (Snapshot) Views

Definition and Operation

A **materialized view**, also known as a snapshot view, is a database object that contains the results of a query, and this result is physically stored in the database. Unlike virtual views, materialized views are actual tables that hold data. The data in a materialized view is a snapshot of the data in the base tables at a specific point in time or the last time the view was refreshed.

Consider the **CustomerOrderCounts** example again. You could create a materialized view based on the same query:

```
CREATE MATERIALIZED VIEW CustomerOrderCountsMaterialized AS
SELECT c.Name, COUNT(o.OrderID) AS NumberOfOrders
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.Name;
```

The first time this statement is executed, the DBMS will run the query and store the resulting data in the **CustomerOrderCountsMaterialized** table. Subsequent queries against this materialized view will directly access the stored data, potentially leading to significant performance improvements.

Benefits of Materialized Views

Materialized views offer distinct advantages, particularly in scenarios with read-heavy workloads:

1. Performance Enhancement:

The primary benefit of materialized views is improved query performance for frequently executed and computationally expensive queries. Since the data is pre-computed and stored, retrieving information from a materialized view is generally much faster than executing the underlying query against the base tables each time. This is especially beneficial for complex analytical queries, data warehousing applications, and scenarios where near real-time data is not strictly required. The concept of optimising data retrieval for efficiency is a key aspect of database design.

2. Hiding Data Complexity:

Similar to virtual views, materialized views can also hide the complexity of underlying data structures and queries from users. Users can query the materialized view as if it were a regular table, without needing to understand its definition's intricate joins or aggregations.

3. Data Consistency (at a Point in Time):

Materialized views provide a consistent view of the data as it existed during the last refresh. This can be advantageous for generating reports or performing analysis that requires a stable and unchanging dataset, even if the underlying base tables are being modified. This "snapshot" characteristic ensures that all queries against the materialized view within a refresh cycle see the same data version.

4. Query Simplification:

By storing the results of complex joins and aggregations, materialized views allow users to write simpler queries to access this pre-processed data. This can improve the usability of the database and reduce the effort required to retrieve commonly needed information.

Considerations for Materialized Views

While offering performance benefits, materialized views also come with certain considerations:

- **Storage Overhead:** Materialized views require physical storage space to store the pre-computed data, unlike virtual views, which only store the query definition.
- **Data Staleness:** The data in a materialized view can become stale as the data in the underlying base tables changes. Mechanisms must be in place to **refresh** the materialized view and bring it up-to-date. Refreshing can be done manually, periodically based on a schedule, or sometimes automatically, depending on the DBMS and configuration.
- **Refresh Costs:** Refreshing a materialised view can be computationally expensive, especially if the underlying query is complex or if there have been significant changes in the base tables. This refresh process can impact the performance of the database system, particularly during peak usage times. Database designers need to consider the refresh strategy and its implications carefully.
- **Write Performance:** Updates to the base tables that are referenced by a materialized view might trigger refresh operations (depending on the refresh strategy), potentially impacting write performance.

SQL Syntax Examples

Virtual View (Generic SQL):

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Materialized View (Syntax may vary depending on the DBMS):

```
CREATE MATERIALIZED VIEW materialized_view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition
WITH DATA | WITH NO DATA; -- (Whether to populate the view immediately)
REFRESH MATERIALIZED VIEW materialized_view_name; -- (Command to manually refresh)
```



Linking Questions and Broader Concepts

Understanding database views connects to several other learning statements within the IB Computer Science HL syllabus:

- **A3.2 Database design:** Views are an integral part of database design, allowing for the logical structuring of data and the implementation of security measures. The choice between virtual and materialized views depends on the application's specific requirements, including data access patterns and performance needs.
- **A3.3 Database programming:** Constructing and querying views involves using SQL, specifically Data Definition Language (DDL) for creating views and Data Manipulation Language (DML) for querying them. Understanding how views interact with SQL queries is essential.
- **Performance considerations:** Performance requirements often drive the choice between virtual and materialized views. Understanding the trade-offs between storage, refresh costs, and query speed is crucial for efficient database programming.
- **Computational Thinking:** Views exemplify the concept of **abstraction** by hiding the underlying complexity of the database schema and presenting a simplified interface to users. They also relate to **decomposition** when a complex data retrieval task is broken down into smaller, manageable view definitions.

Conclusion

Virtualized and materialized database views are essential tools for managing and accessing data effectively. Virtual views offer flexibility, data independence, and security benefits by providing dynamic, logical perspectives on the data. Materialized views, on the other hand, excel in enhancing query performance for read-intensive applications by physically storing pre-computed results. As an IB Computer Science HL student, grasping the nuances of these different view types and their respective advantages and disadvantages will equip you with valuable skills for designing and implementing efficient and secure database systems. Understanding when to use each view type is a key aspect of effective database administration and application development.

Further Exploration

To solidify your understanding, consider the following:

- Design scenarios where using a virtual view would be more appropriate than a materialized view, and vice versa. Justify your choices based on the benefits discussed.
- Explore the specific syntax for creating and managing materialized views in different DBMSs (e.g., MySQL, PostgreSQL, Oracle).
- Consider how views could be used in the context of the annual computer science case study to simplify data analysis or enhance security.
- Consider the ethical implications of using views to control data access and how this relates to data privacy and security.

A 3.3.5 Quiz

Test your knowledge of this section with this quiz:

1

What is the primary difference between a virtual view and a base table?

- A. ☒ Virtual views do not store data physically; they are stored queries.
- B. ☐ Base tables are always read-only, while virtual views can be updated.

- C. ☐ Base tables are defined by queries, while virtual views are not.
- D. ☐ Virtual views store data physically, while base tables do not.

2

Which of the following is a benefit of using virtual views?

- A. ☐ Physical storage of pre-computed results.
- B. ☐ Guaranteed data consistency regardless of base table changes.
- C. ☐ Improved query performance for complex queries.
- D. ☒ Reduced storage overhead compared to base tables.

3

What SQL command is used to create a virtual view?

- A. ☐ CREATE TABLE
- B. ☒ CREATE VIEW
- C. ☐ SELECT INTO
- D. ☐ CREATE MATERIALIZED VIEW

4

Why might a virtual view based on multiple joined tables be read-only?

- A. ☐ Joining tables always results in data inconsistencies upon update.
- B. ☐ Virtual views inherently cannot be updated.
- C. ☒ The DBMS might not be able to determine which base table to update unambiguously.

D. ☐ Read-only access improves the performance of virtual views.

5

Which of the following is a key advantage of materialized views?

A. ☐ They automatically adapt to changes in the underlying base tables without refresh.

B. ☒ They improve query performance for frequently executed queries.

C. ☐ They always reflect the most up-to-date data in real-time.

D. ☐ They do not require any storage space.

6

What is the process of updating the data in a materialized view called?

A. ☐ Committing

B. ☐ Rolling back

C. ☐ Materializing

D. ☒ Refreshing

7

Which of the following is a consideration when using materialized views?

A. ☐ They simplify complex underlying data structures for all queries.

B. ☐ They eliminate the need for indexing on base tables.

C. ☐ They have no impact on write performance on base tables.

D. ☒ The data in them can become stale.

8

The choice between virtual and materialized views is often driven by:

- A. ☐ The complexity of the database schema.
- B. ☐ The specific DBMS being used.
- C. ☒ Performance requirements and data access patterns.
- D. ☐ The number of users accessing the database.

9

How do database views relate to the computational thinking concept of abstraction?

- A. ☐ They decompose complex queries into smaller parts.
- B. ☐ They optimise query execution plans for better performance.
- C. ☐ They ensure data integrity through strict access controls.
- D. ☒ They hide complex underlying structures, presenting a simplified interface.

10

Which SQL Data Definition Language (DDL) command is used to create a materialized view?

- A. ☐ REFRESH VIEW
- B. ☐ CREATE VIEW
- C. ☒ CREATE MATERIALIZED VIEW
- D. ☐ ALTER VIEW

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**